

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 5

Student Name: M.Purna Chandra

Roll No.: A24126552095

Date: 16-08-2026

1. TITLE

Develop a Student Information HTTP Web Server Using Express.js

2. AIM

To create and configure a lightweight Node.js web server using the Express.js framework to handle HTTP GET requests across multiple route endpoints (/, /students, /about) and deliver plain text and structured JSON responses.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the architecture and routing mechanisms of Express.js.
 - To initialize an Express application instance using `express()`.
 - To configure HTTP GET routing handlers using `app.get()`.
 - To send plain text and HTML responses using `res.send()`.
 - To serialize and deliver JSON data structures using `res.json()`.
 - To bind and listen on a dedicated network port (3000) using `app.listen()`.
 - To test and verify API endpoints using web browsers or HTTP clients.
-

4. THEORY

Express.js Framework — Express is a fast, minimalist web framework for Node.js that provides routing, middleware, and HTTP utility methods for building APIs and web servers.

HTTP Routing — Routing defines how an application responds to client requests matching specific endpoints (URI paths) and HTTP request methods (GET, POST, etc.).

app.get(path, handler) — Binds a callback function to handle incoming HTTP GET requests for a designated route path.

res.send(body) — Sends an HTTP response to the client with an automatically set Content-Type header such as text/html.

res.json(data) — Serializes JavaScript objects or arrays into JSON format and sets the Content-Type header to application/json.

app.listen(port, callback) — Binds the server to the specified TCP port and starts listening for incoming client connections.

5. ALGORITHM / PROCEDURE

- Install Express in the project directory using `npm install express`.
 - Create a JavaScript file named `server.js`.
 - Import express using `const express = require("express")`.
 - Initialize an Express app instance: `const app = express()`.
 - Define root GET route `app.get("/", ...)` to send "Welcome to Student Server" via `res.send()`.
 - Define students GET route `app.get("/students", ...)` to return an array of student names in JSON format via `res.json()`.
 - Define about GET route `app.get("/about", ...)` to send "This is a simple Express.js Student Server." via `res.send()`.
 - Start the server on port 3000 using `app.listen(3000, callback)` and log the listening URL.
 - Run the server using `node server.js`.
 - Test requests to `http://localhost:3000/`, `http://localhost:3000/students`, and `http://localhost:3000/about`.
 - Record actual responses against expected outcomes for all routes.
-

6. PROGRAM

server.js

```
const express = require("express");

const app = express();

const students = [
  { id: 1, name: "Rahul", course: "B.Tech" },
  { id: 2, name: "Priya", course: "BCA" },
  { id: 3, name: "Arjun", course: "MBA" },
  { id: 4, name: "Sneha", course: "MCA" },
  { id: 5, name: "Kiran", course: "B.Sc" }
```

```
];

// Home Route
app.get("/", (req, res) => {
  res.send("Welcome to Student Server");
});

// Get All Students
app.get("/students", (req, res) => {
  res.json(students);
});

// About Route
app.get("/about", (req, res) => {
  res.send("This is a simple Express.js Student Server.");
});

// Start Server
app.listen(3000, () => {
  console.log("Server running at http://localhost:3000");
});
```

7. CODE EXPLANATION

Code	Explanation
<code>const app = express()</code>	Creates and initializes an Express application instance.
<code>app.get("/", (req, res) => {...})</code>	Defines an HTTP GET route handler for the root path (/).
<code>res.send("Welcome to Student Server")</code>	Sends a plain text response body to the requesting client.
<code>app.get("/students", ...)</code>	Defines a GET route handler for the /students endpoint.
<code>res.json([...])</code>	Serializes a JavaScript array and returns it with application/json header.
<code>app.listen(3000, ...)</code>	Binds the HTTP server to port 3000 and begins listening for connections.

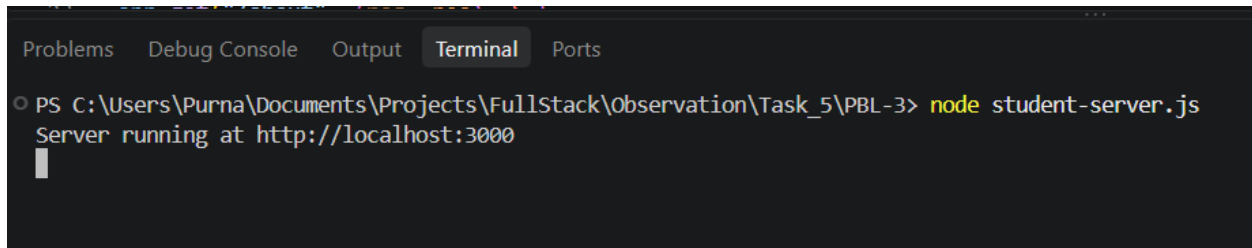
8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Start server with node server.js	Server logs "Server running at http://localhost:3000"	Console message displayed on port 3000	Pass
TC-02	GET request to http://localhost:3000/	HTTP 200 with text "Welcome to Student Server"	Welcome text received successfully	Pass
TC-03	GET request to http://localhost:3000/students	HTTP 200 with JSON array of student names	JSON array returned with 5 students	Pass
TC-04	GET request to http://localhost:3000/about	HTTP 200 with description text	About text returned	Pass
TC-05	GET request to invalid route /unknown	HTTP 404 Cannot GET /unknown	404 error returned as expected	Pass

9. OUTPUT

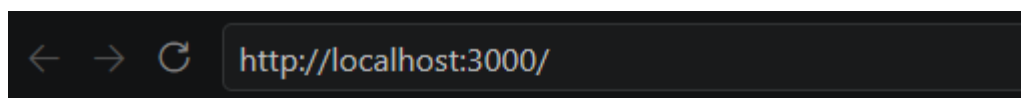
Output 1: Server Startup (Terminal)

Terminal output when starting the Express web server.

A screenshot of a terminal window with a dark background. The terminal shows the command 'node student-server.js' being executed. The output is 'Server running at http://localhost:3000'. The terminal window has tabs for 'Problems', 'Debug Console', 'Output', 'Terminal', and 'Ports'. The 'Terminal' tab is active.

Output 2: Root Route Response (GET /)

Response returned by the server on visiting the root endpoint.



Welcome to Student Server

Output 3: Students API Route Response (GET /students)

JSON response returned on requesting the /students route.

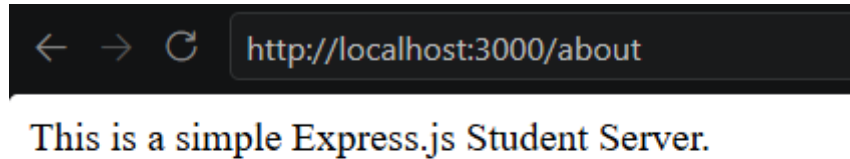


The screenshot shows a web browser window with the address bar displaying `http://localhost:3000/students`. Below the address bar, there is a toggle for "Pretty-print" which is checked. The main content area displays a JSON array of five student objects, formatted for readability. The JSON data is as follows:

```
[
  {
    "id": 1,
    "name": "Rahul",
    "course": "B.Tech"
  },
  {
    "id": 2,
    "name": "Priya",
    "course": "BCA"
  },
  {
    "id": 3,
    "name": "Arjun",
    "course": "MBA"
  },
  {
    "id": 4,
    "name": "Sneha",
    "course": "MCA"
  },
  {
    "id": 5,
    "name": "Kiran",
    "course": "B.Sc"
  }
]
```

Output 4: About Route Response (GET /about)

Response returned by the server on visiting the about endpoint.



Output 5: About Route Response (GET /unknown)

HTTP 404 Cannot GET /unknown.



10. RESULT

Thus, a RESTful HTTP web server was successfully designed and developed using Express.js. The server accurately handles incoming GET requests across routes /, /students, and /about, delivering plain text and JSON responses with proper headers. All test cases passed successfully.